A Low-Code Framework for Conversational AI Assistants Leveraging Retrieval-Augmented Generation and LLMs

[1] Thanigaivel G
Department of Computer Science and Engineering,
Vels Institute of Science Technology and Advanced Studies(VISTAS)
Chennai, India
thanigaivelrubesh1@gmail.com

[2] Thirumal S
Department of Computer Science and Engineering,
Vels Institute of Science Technology and Advanced Studies(VISTAS)
Chennai, India
thirumal.se@vistas.ac.in

[3] Kumar N
Department of Computer Science and Engineering,
Vels Institute of Science Technology and Advanced Studies(VISTAS)
Chennai, India
kumar.se@vistas.ac.in

Abstract* — Conversational AI has evolved rapidly with the recent emergence of large-scale language models, enabling intelligent, context-aware interactions between users and systems across various domains. These models exhibit strong proficiency in interpreting natural language and producing fluent, coherent responses. However, integrating such advanced AI into enterprise applications remains a challenge, often requiring significant programming effort, AI expertise, and system customization. To overcome this limitation, this study introduces a low-code framework for building a domain specific Conversational AI Assistant using Retrieval-Augmented Generation (RAG) and LLMs. The framework is designed to integrate seamlessly with platforms like Oracle APEX, enabling developers and business users to create intelligent assistants with minimal technical complexity. By combining the generative power of LLMs with the precision of retrieval-based methods, the assistant can deliver accurate, real-time responses sourced from enterprise knowledge bases, including documents, databases, and APIs. The framework demonstrates the practical feasibility of deploying conversational AI in enterprise low-code environments.

Keywords* — retrieval augmented generation (RAG), large language model (LLM), low-code application development, Oracle APEX platform, enterprise conversational AI.

I. INTRODUCTION

The fast-paced advancement of conversational artificial intelligence has significantly transformed human–computer interaction by enabling intelligent, context-aware, and natural communication across various domains, including enterprise management, customer service, and education. Recent advancements in Large Language Models (LLMs) have further enhanced the capabilities of conversational agents, allowing them to understand complex queries and generate human-like responses. Despite these advancements, integrating LLM-powered conversational assistants into enterprise workflows remains challenging due to the requirement for extensive programming, specialized AI expertise, and complex system integration.

Conventional chatbot systems, which are typically rule-based or template-driven, lack the flexibility and contextual awareness required for dynamic enterprise environments. Such systems struggle to retrieve up-to-date domain knowledge and often fail to provide accurate responses for complex, domain-specific queries. To address these challenges, Retrieval-Augmented Generation (RAG) has been introduced as a practical solution that improves LLMs by incorporating external knowledge sources. By combining information retrieval techniques with generative models, RAG enables conversational systems to produce responses that are both contextually relevant and grounded in enterprise data.

While RAG-based conversational AI systems have demonstrated promising results, their adoption in enterprise applications is limited by the complexity of development and deployment. Most existing implementations rely on custom-coded pipelines and infrastructure, making them difficult to integrate into low-code enterprise platforms. This creates a gap between advanced AI capabilities and their practical utilization within enterprise application development environments.

To overcome this research gap, this work introduces a low-code framework for designing enterprise-oriented conversational AI assistants based on retrieval-augmented generation in combination with large-scale language models, integrated with Oracle APEX. The proposed framework leverages the low-code paradigm to reduce development complexity and facilitate smooth incorporation of conversational AI within established enterprise environments. The proposed framework combines natural language understanding, knowledge retrieval, and response generation within a unified and scalable architecture, making it accessible to both developers and non-technical users.

The effectiveness of the proposed framework is evaluated in terms of integration feasibility, usability, and response relevance in an enterprise-like environment. The results indicate that the low-code approach simplifies the development process while ena bling the deployment of context-aware conversational AI assistants suitable for enterprise applications.

II. RELATED WORK

Conversational artificial intelligence has gained growing research interest in recent years, driven by advancements in large language models and their capability to produce fluent and contextually appropriate responses. Early conversational systems were primarily rule-based or retrieval-driven, which limited their adaptability and effectiveness in handling complex, domain-specific queries. These limitations motivated the integration of LLMs with knowledge retrieval strategies to enhance both system adaptability and factual reliability.

Numerous studies have investigated the application of LLM-driven conversational agents in domain-specific contexts. Yin and Decker [1] demonstrated the adoption of LLM-driven chatbot systems in academic institutions, showing that domain knowledge integration significantly improved the accuracy of responses to database-related queries. Similarly, Sarferaz [2] investigated the deployment of generative AI within enterprise resource planning (ERP) systems and highlighted the importance of contextual awareness for automating business processes.

The emergence of Retrieval-Augmented Generation (RAG) has further enhanced conversational AI by combining information retrieval with generative language models. Hybrid architectures leveraging RAG have been shown to reduce hallucination and improve response reliability. Yung et al. [3] proposed a scalable RAG pipeline integrated with vector search techniques for large-scale document retrieval, reporting improved performance in real-time enterprise query handling. Additionally, recent advances in embedding-based retrieval methods have contributed to better contextual grounding across diverse domains [14].

In parallel, low-code and no-code platforms have gained popularity for accelerating application development in enterprise environments. Platforms such as Oracle APEX and Microsoft Power Platform provide simplified development workflows that reduce technical barriers and enable rapid deployment of AI-enabled applications. However, existing studies indicate that these platforms offer limited native support for integrating advanced LLM-based conversational systems with domain-specific knowledge bases, often requiring custom extensions or external services.

Recent frameworks such as LlamaIndex [12] and LangChain [13] have facilitated the integration of vector databases with LLMs, enabling efficient semantic retrieval for context augmentation. While these frameworks demonstrate strong potential for building RAG-based conversational agents, they typically require substantial programming expertise and infrastructure management, which limits their accessibility for non-technical users and low-code environments.

Based on the reviewed literature, it is evident that although significant progress has been made in LLM-based conversational AI and RAG architectures, there remains a gap in developing enterprise-ready conversational assistants that can be easily configured and deployed using low-code platforms. This work addresses this gap by proposing a low-code framework that integrates LLMs with Retrieval-Augmented Generation, focusing on accessibility, modularity, and seamless integration with enterprise systems such as ERP and CRM platforms.

III. PROPOSED METHODOLOGY

This section presents a low-code AI framework that combines large language models with retrieval-augmented generation (RAG) to deliver context-aware and domain-specific responses in enterprise environments. The framework is designed to minimize development complexity using low-code platforms while ensuring scalability, accuracy, and seamless integration with organizational workflows. The system's overall process flow is depicted in the Fig. 1.

A. System Architecture

The system architecture consists of a low-code user interface, a middleware processing layer, a retrieval engine, and an LLM-based response generator. Users interact with the system through a chat interface developed using low-code platforms such as Oracle APEX. The middleware layer handles query preprocessing, embedding generation, and orchestration of retrieval and generation tasks. Enterprise knowledge sources, including structured databases,ERP systems, and unstructured documents, are stored in a vector database to enable efficient semantic retrieval. The retrieved context is combined with user queries and processed by the LLM to generate accurate and domain-relevant responses.

B. User Query Handling

The interaction starts when a user inputs a natural language query via the chat-based interface. The interface is designed to be intuitive and accessible, allowing users to interact with enterprise systems without requiring technical expertise. The submitted query is forwarded to the middleware layer, where it undergoes preprocessing before being passed to the embedding and retrieval modules.

C. Embedding Generation

To enable semantic understanding of user input, the system converts each query into a high-dimensional vector representation using pretrained embedding models such as OpenAI Embeddings or Sentence-BERT. This representation captures the contextual meaning and intent of the query, allowing the system to perform effective similarity matching against enterprise knowledge sources.

D. Knowledge Retrieval Using RAG

The Retrieval-Augmented Generation (RAG) module enhances response accuracy by dynamically retrieving relevant enterprise knowledge.

1. Document Processing

Enterprise knowledge sources, including ERP data, structured databases, and unstructured documents, are preprocessed by cleaning and segmenting them into manageable text chunks. These chunks are converted into vector embeddings using the same embedding model employed for query encoding and stored in a vector database like FAISS to facilitate efficient similarity-based retrieval.

2. Query Embedding

Upon receiving a user query, it is transformed into a vector representation using the embedding model. A similarity search is conducted on the vector database to retrieve the most relevant documents. or records corresponding to the query.

3. Context Injection

The retrieved documents are selected based on similarity scores and combined with the user's query. This retrieved context serves as supporting evidence that grounds the response generation process and improves factual accuracy.

4. Augmented Prompt Construction

The retrieved information is combined with the user query to create an augmented prompt. This prompt provides the LLM with both the user's intent and relevant domain-specific context, thereby reducing hallucinations and ensuring that responses align with organizational data and policies.

E. LLM Response Generation

The augmented prompt is processed by an advanced LLM such as GPT-4 to generate a fluent, coherent, and context-aware response. By leveraging retrieved enterprise knowledge, the model produces responses that are both linguistically natural and factually grounded, making them suitable for enterprise use cases.

F. Integration and Deployment

The generated response is delivered back to the user in real time through the chat interface. User interactions and feedback are logged to support continuous improvement of retrieval accuracy and prompt optimization. The framework is deployed within low-code environments such as Oracle APEX, enabling rapid integration with existing enterprise workflows including ERP and CRM systems. The modular design ensures scalability, maintainability, and accessibility for both developers and non-technical users.

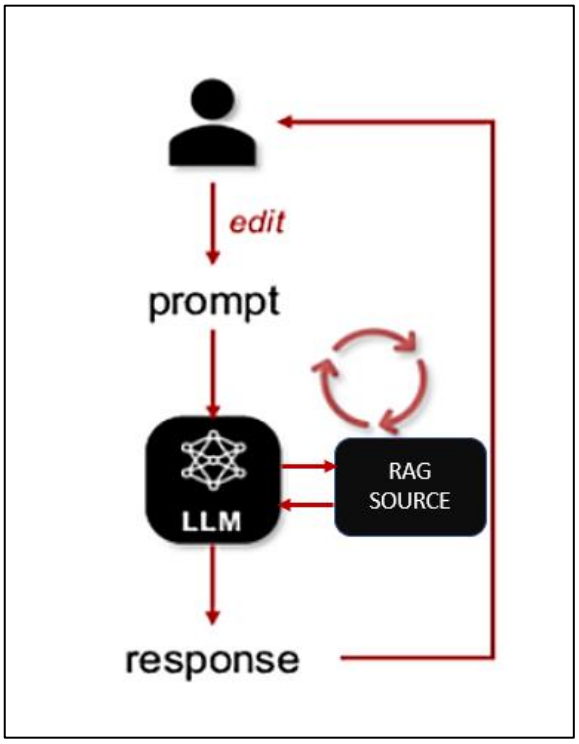

Fig. 1. Module Flow Diagram

IV. IMPLEMENTATION AND EVALUATION

. This section outlines the complete implementation methodology for the proposed low-code conversational AI framework and evaluates its effectiveness in an enterprise-oriented environment. The implementation focuses on seamless integration, ease of development, and practical usability rather than heavy infrastructure complexity.

A. Implementation Details

The proposed framework was implemented using a low-code development approach to demonstrate ease of integration and rapid deployment. The user interface was developed using Oracle APEX, providing a chat-based interaction layer that enables users to submit natural language queries. The middleware layer was implemented using RESTful APIs to handle query processing, embedding generation, and communication with external AI services.

Enterprise knowledge sources, including structured records and unstructured documents, were preprocessed and indexed into a FAISS-based vector

database. Embedding generation was performed using pretrained embedding models such as OpenAI Embeddings or Sentence-BERT to ensure semantic consistency between indexed data and user queries. The LLM component (e.g., GPT-4) was accessed through secured API endpoints to generate responses based on augmented prompts.

The system was deployed in a modular manner, allowing individual components such as retrieval, prompt construction, and response generation to be updated independently. This modularity supports scalability and simplifies maintenance within enterprise environments.

B. Experimental Setup

To assess the performance of the proposed framework, a set of domain-specific queries was prepared based on typical enterprise use cases, including ERP-related questions, document-based information retrieval, and general business queries. These queries were executed through the conversational interface, and responses were generated using the RAG-enabled LLM pipeline.

The evaluation environment consisted of:

- A low-code frontend developed in Oracle APEX
- A vector database storing enterprise knowledge embeddings
- An LLM accessed via API for response generation

The system was tested under normal operational conditions to assess response relevance, contextual accuracy, and system usability.

C. Evaluation Metrics

The proposed system's performance was assessed using both qualitative and quantitative metrics. commonly adopted in conversational AI research:

- **Response Accuracy**: Degree to which generated responses matched enterprise knowledge.
- **Contextual Relevance**: Ability of the system to maintain relevance to the user query.
- **Development Effort**: Reduction in coding complexity achieved through low-code integration.
- **Response Latency**: Time taken to generate responses after query submission.
- **Usability**: Ease of interaction for non-technical users.

D. Results and Discussion

Experimental results indicate that the proposed RAG-enabled conversational assistant produces more accurate and contextually relevant responses compared to traditional rule-based chatbots. The integration of retrieval mechanisms significantly reduced hallucinations and ensured alignment with enterprise data sources.

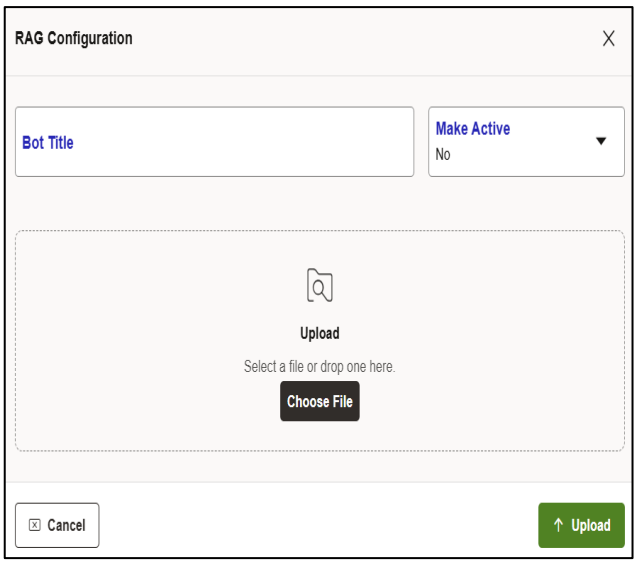

Fig. 2. Admin RAG Configuration

The lowcode implementation exhibited a significant reduction in the development effort., enabling faster deployment and easier customization compared to conventional AI system development. Response latency remained within acceptable limits for real-time interaction, making the system suitable for enterprise applications such as ERP and CRM support.

Overall, the results validate that combining LLMs with Retrieval-Augmented Generation in a low-code environment provides an effective and practical solution for building enterprise-ready conversational AI assistants.

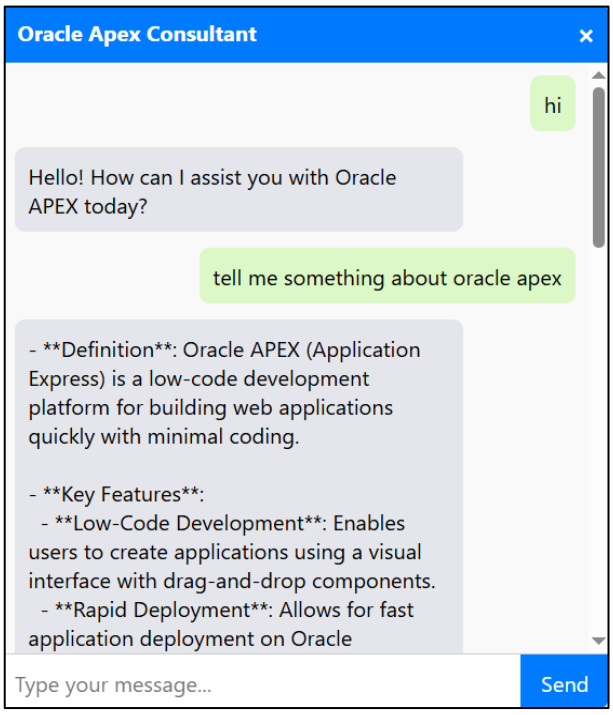

Fig. 3. RAG Powered Chatbot

User interaction logs further indicated that the RAG-based assistant reduced repeated queries and clarification requests, demonstrating improved response clarity. The system also maintained consistent performance across different query categories, including structured database queries and unstructured document queries. These observations confirm the robustness of the retrieval-augmented generation strategy in handling heterogeneous enterprise knowledge sources.

E. Limitations

Despite its effectiveness, the proposed framework has certain limitations. The quality of generated responses is strongly reliant on the precision of the retrieved documents. Incorrect or outdated knowledge sources may affect response reliability. Additionally, large-scale document collections may require optimized indexing strategies to maintain retrieval performance. Real-time embedding updates also introduce computational overhead in dynamic enterprise environments.

V. Comparative Analysis and Enterprise Impact

In order to further evaluate the performance of the proposed low-code RAG-based conversational AI framework, a comparative analysis was performed against traditional rule-based chatbots and direct LLM-only conversational systems. The comparison focused on response accuracy, contextual relevance, development complexity, and enterprise adaptability.

Traditional rule-based systems demonstrated fast response times but failed to handle complex and context-dependent queries. In contrast, direct LLM-based chatbots produced fluent responses but occasionally generated hallucinated or unsupported information due to the absence of grounding enterprise knowledge. The proposed RAG-enabled framework successfully balanced both approaches by combining semantic retrieval with generative reasoning, thereby producing responses that were both linguistically natural and factually reliable.

From a development perspective, conventional chatbot development required significant manual coding and maintenance effort. The low-code integration using Oracle APEX significantly reduced development time, allowing rapid configuration of user interfaces, data sources, and conversational workflows. This advantage enables non-technical stakeholders to participate in chatbot customization, improving enterprise adoption.

In terms of enterprise impact, the proposed system demonstrated strong applicability for ERP support, document retrieval, and internal knowledge assistance. Employees were able to retrieve accurate operational information without navigating complex enterprise portals, thereby improving productivity and reducing support workload.

The modular architecture further allows organizations to extend the system with minimal changes to existing workflows. Compared to fully custom AI systems, the proposed framework offers a practical and cost-effective solution for enterprises seeking scalable conversational AI deployment. Moreover, the low-code design significantly reduces long-term maintenance effort and enables rapid adaptation to evolving business requirements. This flexibility makes the framework suitable for continuous enterprise digital transformation initiatives without introducing additional technical complexity.

VI. CONCLUSION

This paper presented a low-code framework for building enterprise-grade conversational AI assistants by combining large language models (LLMs) with retrieval-augmented generation (RAG). The proposed approach enables intelligent, context-aware, and domain-specific conversational systems while significantly reducing development complexity through low-code platforms such as Oracle APEX. By combining semantic retrieval with generative language models, the framework ensures accurate and reliable responses aligned with enterprise knowledge sources.

The implementation and evaluation results demonstrate that the proposed system improves response accuracy and contextual relevance when compared to traditional rule-based chatbots. Additionally, the low-code architecture facilitates rapid deployment and seamless integration with existing enterprise workflows such as ERP and CRM systems, making the framework practical for real-world enterprise applications.

Furthermore, the proposed framework highlights how low-code platforms can serve as effective enablers for scalable and maintainable conversational AI deployment in enterprise environments. By abstracting architectural complexity, the system allows organizations to focus on business objectives rather than technical implementation challenges. This work therefore contributes both a practical architectural solution and experimental insights that support the adoption of Retrieval-Augmented Generation in enterprise conversational intelligence.

VII. FUTURE WORK

Future enhancements to the proposed framework include the integration of multimodal interaction capabilities such as voice-based input and real-time language translation to enable more natural communication. The system can also be extended into a multi-agent architecture capable of handling complex enterprise tasks such as scheduling, reporting, and automated data updates.

Furthermore, incorporating continuous learning mechanisms based on user feedback and cloud-native deployment strategies can improve scalability, adaptability, and performance. Domain-specific

optimizations for sectors such as healthcare, finance, and supply chain management also present promising directions for extending the applicability of the proposed framework.

Moreover, future work may investigate privacy-preserving conversational AI techniques, including on-premise deployment and federated learning approaches, to support organizations with strict data governance requirements. Another important direction is the evaluation of energy efficiency and cost optimization strategies for large-scale enterprise deployments.

The integration of explainable AI mechanisms can further improve user trust by providing transparent reasoning behind generated responses. Additionally, benchmarking the proposed framework against emerging enterprise conversational AI platforms will help establish standardized performance comparisons and strengthen its practical relevance.

In addition, future research may explore adaptive retrieval strategies that dynamically adjust document selection based on query complexity and historical interaction patterns. The integration of fine-tuned, domain-specific LLMs can further enhance response precision in specialized enterprise environments. Finally, large-scale longitudinal evaluations involving real enterprise users will provide deeper insights into system reliability, user acceptance, and long-term operational impact.

VIII. REFERENCES

[1] Y. Yin and S. Decker, "LLM-based chatbot applications in higher education for databases and information systems," *Proc.IEEE Conf. on Artificial Intelligence in Education*, 2025.

[2] S. Sarferaz, "Implementing generative AI into ERP software," *Proc. IEEE Int. Conf. on Enterprise Systems*, 2025.

[3] R. Yang *et al.*, "Engineering retrieval augmented generation based virtual assistants in practice," *arXiv preprint*, 2025.

[4] C. Jeong, "Beyond text: Implementing multimodal LLM-powered multi-agent systems using a no-code platform," *Proc. IEEE Int. Conf. on Artificial Intelligence*, 2025.

[5] S. Ghosh and G. Mittal, "Low code RAG & LLM framework for the context aware querying in the electrical standards, design and research " *Preprints*, 2025.

[6] O. Marquez Ayala and P. Béchard, "Generating a low-code workflow via task decomposition and retrieval-augmented generation," *Proc. IEEE Int. Conf. on Software Engineering Workshops*, 2024.

[7] Y. Cai, Z. Xing, *et al.*, "White-box LLM-supported low-code engineering: A vision and first insights," *Proc. ACM/IEEE MODELS Companion*, 2024.

[8] IEEE Software Editorial Board, "The next frontier in software development: AI-augmented software development processes," *IEEE Software*, volume. 40 , No. 3, PP. 5 - 8, 2023.

[9] J. Terven, A.Arguelles, and I.Cervantes, "Intelligent driving assistance system using retrieval-augmented generation," *Proc.IEEE International. Conference on Intelligent Transportation Systems*, 2024.

[10] S. Aboukadri and I. El Asri, "Leveraging retrieval-augmented generation and large language models for access control policy extraction from user stories in agile software development," *Proc. IEEE Int. Conf. on Software Engineering*, 2025.

[11] The Oracle, "The Oracle APEX document," [Official Website]. Available: https://docs.oracle.com/en/database/oracle/apex/

[12] LlamaIndex, "LlamaIndex document," [Official Website]. Available: https://llamaindex.ai

[13] Lang-Chain, "The Lang-Chain document," [Official Webiste]. Available: https://docs.langchain.com

[14] OpenAI, "Embeddings API document," [Official Webiste]. Available: https://platform.open ai.com/docs/guides/embeddings

[15] B.Tural, Z.Örpek, and Z.Destan, "Retrieval augmented generation (RAG) & LLM integration," in *2024 8th International Symposiym. on Innovative Approaches in Smart Technologies (ISAS)*, IEEE , 2024.

[16] B. Shelley, P. Kaur, G. Aggarwal, A. Kaushal, and M. K. Dutta, "Flora-RAG: Enhancing conversational AI with retrieval augmented generation for floriculture," in *2025 Int. Conf. on Engineering, Technology & Management (ICETM)*, IEEE, 2025.

[17] B.Sawarkar, A.Mangal, and S.R.Solanki, "Blended RAG : Improving retrieval augmented generation accuracy, with semantic search and hybrid query based retrievers," *in the 2024 IEEE 7th Int. Conf. on the Multimedia Information Processing & Retrieval (MIPR)*, IEEE, 2024.

[18] C. Cai, S. Chen, Y. Wu, Y. Huang, J.Feng, and Z.Ou, "The 2nd Futuredial challenge :Dialog systems with the retrieval augmented generation (Futuredial RAG)," in *2024 IEEE Spoken Language Technology Workshop-(SLT)*, IEEE, 2024.